Design_Patterns

1. Strategy
    1. Strategy denotes a pattern that allows control over variability of algorithms. Its integral part constitutes an interface that allows uniform application of different algorithms.
    2. The algorithms can be defined in many ways, for instance as function objects.
    3. A particular algorithm is chosen based on some information on processed data.
2. Proxy
    1. The role of the proxy object is to become a **placeholder** or **surrogate** in place of another object.
    2. An external caller cannot tell a proxy from its counterpart. However, to be useful the proxy has to have some advantages over an object to which it is a surrogate. Usually, this is its smaller size or deferred implementation, etc.
3. Factory method
    1. In some situations we need an interface for creating objects of some hierarchy but a choice of the particular one is left to some classes in another hierarchy. In such a situation a factory method design pattern, also called a virtual constructor pattern, can be of help.
4. Prototype
    1. Sometimes we are interested in creating a copy of an object accessed by a reference or pointer to its base class, however. Hence, we cannot easily tell which particular derivative of a base we have accessed. One method is to use the C++ run-time-type-information (RTTI) mechanism. However, this results in not very friendly switch-case statements.
    2. The other way is to use a key mechanism of the prototype design pattern, namely the virtual **clone()** method, defined for each class in a hierarchy. This method is responsible for creating and returning a new object of exactly the same type as its object and with exactly the same state as its object. In each clone() this is implemented simply by calling the new operator and copy constructor of the class to which clone belongs.
    3. example:

```
TCoordTransform_Proto* TLinearTransform_Proto::Clone(void):
{
    return new TLinearTransform_Proto(*this);  // copy constructor
}
```

    4. example:

```
unique_ptr<Bullet> clone() override:
{
    return make_unique<SimpleBullet>(*this);
}
```